

Experiment No.01

Name of Experiment:

Perform a case study by installing and exploring various types of operating systems on a physical or logical (virtual) machine. (Linux Installation).

Problem Statement: Installation of Fedora 42 Workstation on physical Machine

Software /Hardware Requirements: Fedora 42 Workstation ISO Image Pen Drive (Bootable) ,PC

Theory:

1. Introduction to Linux

Linux is a powerful, open-source, UNIX-like operating system originally developed by **Linus Torvalds** in 1991. It has since grown into one of the most widely used OS kernels, forming the basis for various **distributions (distros)** like Ubuntu, Fedora, Debian, Arch, Red Hat, etc.

Linux is widely used in:

- Servers
- Desktops
- Embedded Systems
- Mobile Devices (e.g., Android)
- Supercomputers
- IoT Devices

2. Key Features of Linux

Feature	Description
Open Source	Source code is freely available and modifiable
Multiuser System	Multiple users can use system resources simultaneously
Multitasking	Can run multiple processes concurrently
Portability	Can run on a wide range of hardware platforms
Security	Offers user permissions, firewalls, SELinux, etc.
Stability	Rarely needs to be rebooted; ideal for servers
Modularity	Components like kernel, shell, filesystem are independent
Shell Interface	Command-line interface to control the system

3. Components of Linux Operating System

A. Kernel

- Core component managing hardware and system resources
- Types: Monolithic (Linux), Microkernel, Hybrid
- Responsibilities: Process scheduling, memory management, device drivers

B. Shell

- Interface between user and kernel
- Examples: Bash (default), Zsh, Fish
- Interprets commands entered by users

C. File System

- Everything in Linux is treated as a file
- Standard hierarchy starts from / (root)
- Common directories: /home, /etc, /bin, /var, /dev

D. System Utilities

- Essential tools and programs for managing the system
- Example: ls, cd, cp, ps, top, kill

E. Application Programs

- Graphical and text-based software like browsers, editors, IDEs

4. Linux Distributions (Distros)

A Linux **distribution** is a complete operating system built around the Linux kernel, with added tools and package managers.

Distribution	Key Features
Ubuntu	User-friendly, great for beginners
Fedora	Cutting-edge features, Red Hat-backed
Debian	Very stable and community-driven
Arch	Lightweight, rolling release, advanced users
CentOS/AlmaLinux	Server-based, enterprise-ready

5. Linux File System Hierarchy

Linux follows a standard file system layout:

/	
— bin/	→ Essential binaries
— boot/	→ Bootloader files
— etc/	→ Configuration files
— home/	→ User home directories
— root/	→ Root user's home
— usr/	→ User programs and data
— var/	→ Variable data like logs, mail

6. Common Linux Commands

Command	Purpose
ls	List files/directories
cd	Change directory
pwd	Print current directory
cp, mv	Copy or move files
rm	Remove files
chmod	Change file permissions
top, htop	Show running processes
sudo	Run command with super user privileges
dnf, apt	Install and manage packages

7. Package Management

- Linux uses **package managers** to install, update, and remove software.
- Common Package Managers:
 - **APT**: Debian-based (Ubuntu)
 - **DNF/YUM**: Red Hat-based (Fedora, CentOS)
 - **Pacman**: Arch Linux
 - **Flatpak/Snap**: Universal formats for all distros

8. Applications of Linux

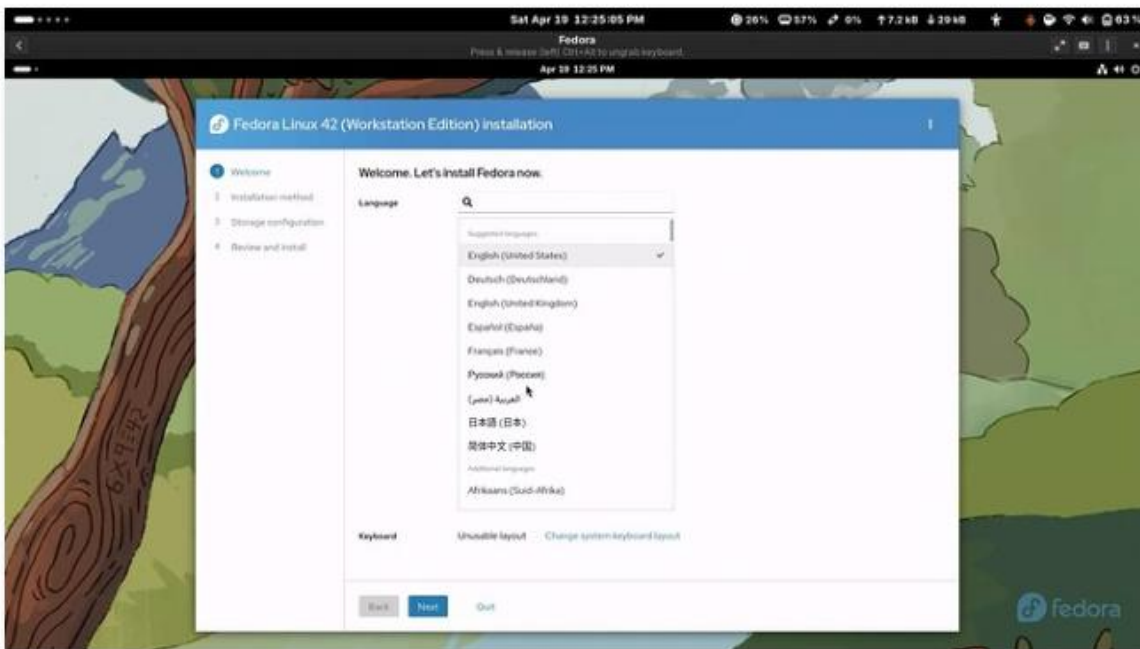
- **Servers**: Web, file, DNS, email servers
- **Software Development**: Supports a wide range of programming languages and tools
- **Networking**: Routing, switching, network analysis
- **Cybersecurity**: Tools like Kali Linux for penetration testing
- **Education and Research**
- **Cloud Platforms**: AWS, Azure use Linux extensively

9. Advantages of Linux

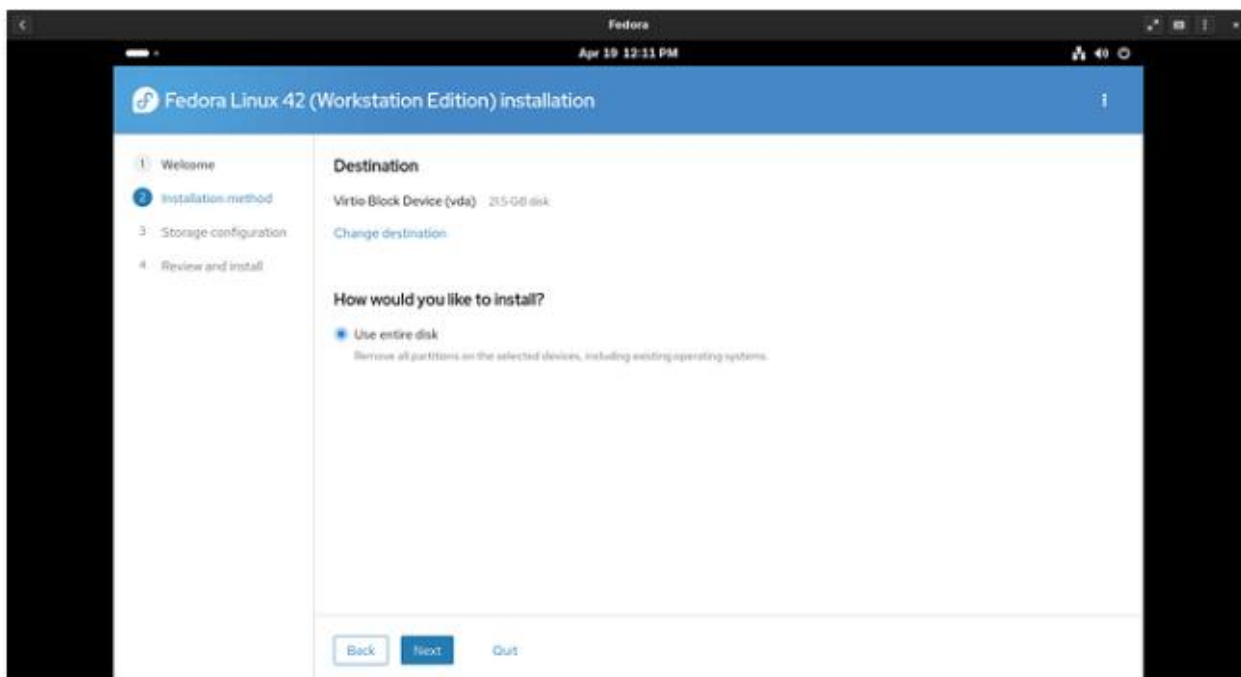
- Free and open-source
- Highly customizable
- Secure and stable
- Great community support
- Large software repositories

Fedora 42 Workstation Installation Process:

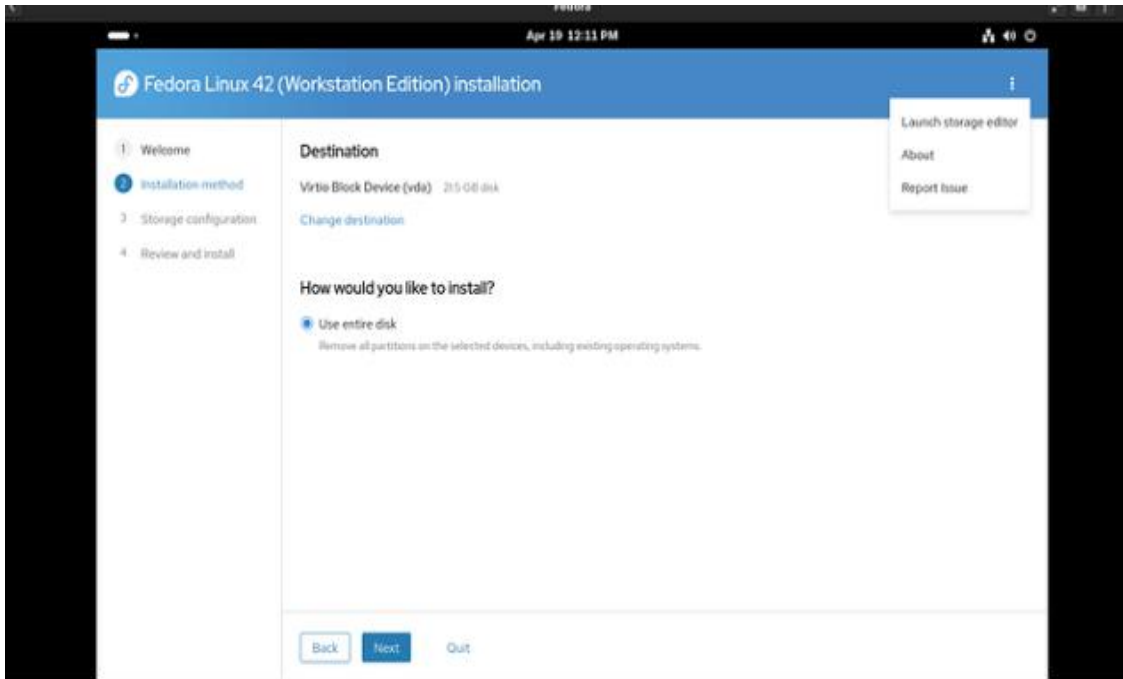
1. Boot into fedora 42 using live iso using your preferred way of flashing the iso
2. Start the installer and select you language and keyboard layout



3. Here you will be listed with available partition, go to the top right 3 dot



4. From the 3 dot select **Launch storage editor**



5. Create the Free Space on the Disk by Shrink the Size
6. Linux required 40 GB of Free Space
7. Install the Setup and finished the installation

To dual-boot Fedora 42 and Windows, you'll need to prepare your system by creating free space for Fedora, creating a bootable USB drive with Fedora 42, and then installing Fedora, making sure to choose the correct partitioning scheme during installation.

Here's a more detailed breakdown:

1. Preparation:

- **Backup:**

Back up your important data from your Windows system, as partitioning can be risky.

- **Shrink Windows Partition:**

In Windows, use Disk Management to shrink the existing Windows partition and create unallocated space for Fedora. You'll need at least 40GB for Fedora.

- **Download Fedora:**

Download the Fedora Workstation 42 ISO file from the Fedora Project website.

- **Create Bootable USB:**

Use a tool like Rufus (or Fedora Media Writer, which is recommended) to create a bootable USB drive from the ISO file. Ensure the partition scheme is set to GPT.

2. Installation:

- **Boot from USB:** Boot your computer from the Fedora bootable USB drive.
- **Start Installation:** Launch the Fedora installer and choose your language and keyboard layout.

- **Choose Installation Destination:** Select the unallocated space you created earlier as the installation destination. You can choose the automatic partitioning option, or you can create custom partitions if you prefer.
- **Partitioning (if custom):** If you choose custom partitioning, you'll need to create at least three partitions: /boot/efi (for the UEFI bootloader), /boot (for the kernel and other boot files), and / (the root partition). You might also want to create a /home partition for your user files.
- **Encryption (optional):** You can choose to encrypt your Fedora installation for added security.
- **Review and Install:** Review your installation settings and click "Begin Installation".
- **Final Setup:** Once the installation is complete, you'll be guided through the final setup steps.

3. Dual-Boot Configuration:

- **GRUB Bootloader:**

Fedora's installer will configure the GRUB bootloader to allow you to choose between Windows and Fedora at start up.

- **BIOS/UEFI settings:**

In some cases, you may need to enter your computer's BIOS/UEFI settings and adjust the boot order to prioritize GRUB or Fedora if it doesn't automatically boot into the GRUB menu.

Important Considerations:

- **Secure Boot:**

If your computer has secure boot enabled, you might encounter issues during installation or booting. You may need to disable secure boot in your BIOS/UEFI settings, or ensure Fedora is properly signed to work with secure boot.

- **Disk Partitioning:**

Carefully review the partitioning steps to avoid accidentally deleting your Windows installation.

- **Troubleshooting:**

If you encounter any issues during the installation process, consult the Fedora documentation or seek help from the Fedora community.

Conclusion:

In this Way we successfully install the Fedora 42 Workstation on the Personal Computer.

Experiment No.02

Name of Experiment: Implement of different types of system calls.

Problem Statement: Implement the fork () and Write () system call in Linux operating System

Software /Hardware Requirements: GCC Compiler Linux Operating System

Theory:

A **system call** is a programmatic way in which a computer program requests a service from the kernel of the operating system on which it is executed. A system call is a way for programs to **interact with the operating system**. A computer program makes a system call when it requests the operating system's kernel. System call **provides** the services of the operating system to the user programs via the Application Program Interface (API). System calls are the only entry points into the kernel system and are executed in kernel mode.



- A user program can interact with the operating system using a system call. A number of services are requested by the program, and the OS responds by launching a number of systems calls to fulfill the request.
- A system call can be written in high-level languages like C or Pascal or in assembly language. If a high-level language is used, the operating system may directly invoke system calls, which are predefined functions.
- A system call is initiated by the program executing a specific instruction, which triggers a switch to kernel mode, allowing the program to request a service from the OS. The OS then handles the request, performs the necessary operations, and returns the result back to the program.
- System calls are essential for the proper functioning of an operating system, as they provide a standardized way for programs to access system resources. Without system calls, each program would need to implement its methods for accessing hardware and system services, leading to inconsistent and error-prone behavior.

Services Provided by System Calls

- Process Creation and Management
- Main Memory Management
- File Access, Directory, and File System Management
- Device Handling(I/O)
- Protection
- Networking, etc.
- **Process Control:** end, abort, create, terminate, allocate, and free memory.
- **File Management:** create, open, close, delete, read files, etc.

- **Device Management**
- **Information Maintenance**
- **Communication**

Features of System Calls

- **Interface:** System calls provide a well-defined interface between user programs and the operating system. Programs make requests by calling specific functions, and the operating system responds by executing the requested service and returning a result.
- **Protection:** System calls are used to access privileged operations that are not available to normal user programs. The operating system uses this privilege to protect the system from malicious or unauthorized access.
- **Kernel Mode:** When a system call is made, the program is temporarily switched from user mode to kernel mode. In kernel mode, the program has access to all system resources, including hardware, memory, and other processes.
- **Context Switching:** A system call requires a context switch, which involves saving the state of the current process and switching to the kernel mode to execute the requested service. This can introduce overhead, which can impact system performance.
- **Error Handling:** System calls can return error codes to indicate problems with the requested service. Programs must check for these errors and handle them appropriately.
- **Synchronization:** System calls can be used to synchronize access to shared resources, such as files or network connections. The operating system provides synchronization mechanisms, such as locks or semaphores, to ensure that multiple programs can access these resources safely.

How does System Call Work?

Here is a detailed explanation step by step how system calls work:

- **Users need special resources:** Sometimes programs need to do some special things that can't be done without the permission of the OS like reading from a file, writing to a file, getting any information from the hardware, or requesting a space in memory.
- **The program makes a system call request:** There are special predefined instructions to make a request to the operating system. These instructions are nothing but just a "system call". The program uses these system calls in its code when needed.
- **Operating system sees the system call:** When the OS sees the system call then it recognizes that the program needs help at this time so it temporarily stops the program execution and gives all the control to a special part of itself called 'Kernel'. Now 'Kernel' solves the need of the program.
- **The operating system performs the operations:** Now the operating system performs the operation that is requested by the program. Example: reading content from a file etc.
- **Operating system give control back to the program :** After performing the special operation, OS give control back to the program for further execution of program.

Examples of a System Call in Windows and Unix

System calls for Windows and Unix come in many different forms. These are listed in the table below as follows:

Process	Windows	Unix
Process Control	CreateProcess() ExitProcess() WaitForSingleObject()	Fork() Exit() Wait()
File manipulation	CreateFile() ReadFile() WriteFile()	Open() Read() Write() Close()
Device Management	SetConsoleMode() ReadConsole() WriteConsole()	Ioctl() Read() Write()
Information Maintenance	GetCurrentProcessID() SetTimer() Sleep()	Getpid() Alarm() Sleep()
Communication	CreatePipe() CreateFileMapping() MapViewOfFile()	Pipe() Shmget() Mmap()
Protection	SetFileSecurity() InitializeSecurityDescriptor() SetSecurityDescriptorGroup()	Chmod() Umask() Chown()

Program Code:

```
/* C Program to Implement fork() System Call */
```

```
#include <stdio.h>
```

```
#include <unistd.h>
```

```
#include <sys/types.h>
```

```
int main() {
```

```
    pid_t pid;
```

```
    // Create a new process
```

```
    pid = fork();
```

```
    // Check for error
```

```
    if (pid < 0) {
```

```
        perror("Fork failed");
```

```
        return 1;
```

```
}

// Child process block
if (pid == 0) {
    printf("Child Process:\n");
    printf("Child PID: %d\n", getpid());
    printf("Parent PID (from child): %d\n", getppid());
}
// Parent process block
else {
    printf("Parent Process:\n");
    printf("Parent PID: %d\n", getpid());
    printf("Child PID (from parent): %d\n", pid);
}

return 0;
}
```

Output:

Compilation and Execution (Linux terminal)

1. **Save the file** as `fork_example.c`
2. **Compile the program** using GCC:

```
gcc fork_example.c -o fork_example
```

3. **Run the executable:**

```
./fork_example
```

```
Parent Process:
Parent PID: 12345
Child PID (from parent): 12346
Child Process:
Child PID: 12346
Parent PID (from child): 12345
```

/* C Program to Implement write() System Call */

```
#include <unistd.h>    // for write()
#include <fcntl.h>      // for open() and O_* constants
#include <sys/types.h>
#include <sys/stat.h>
#include <stdio.h>      // for perror()

int main() {
    int fd;
    const char *text = "Hello from write() system call in C!\n";

    // Open a file (or create it if it doesn't exist)
    fd = open("output.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
```

```
if (fd < 0) {
    perror("File open failed");
    return 1;
}

// Write to the file
ssize_t bytes_written = write(fd, text, 37); // write 37 bytes

if (bytes_written < 0) {
    perror("Write failed");
    close(fd);
    return 1;
}

printf("Successfully written %ld bytes to output.txt\n", bytes_written);

// Close the file
close(fd);

return 0;
}
```

- Save the code as `write_example.c`
- Compile with `gcc`:

```
gcc write_example.c -o write_example
```

- Run the program:

```
./write_example
```

- Check the contents of `output.txt`:

```
cat output.txt
```

Conclusion: In this way we implement `fork()` and `Write()` system call in operating system

Experiment No.03

Name of Experiment: Write a program to implement FCFS, SJF and priority based Algorithm.

Problem Statement: Implement a C program for FCFS SJF and Priority based Algorithm

Software /Hardware Requirements: Turbo C++ Compiler Personal Computer

Theory:

SJF Algorithm

Shortest Job First (SJF) is a non-preemptive scheduling algorithm that selects the process with the shortest burst time (execution time) to execute next. It is also known as Shortest Job Next (SJN).

Working Principle:

- At any scheduling point, the CPU is assigned to the process with the smallest burst time.
- If two processes have the same burst time, FCFS (First Come First Serve) is used as a tie-breaker.
- Once a process starts execution, it runs to completion (non-preemptive).

Steps:

1. Sort all processes by **burst time**.
2. Execute the process with the **shortest burst time**.
3. Calculate:
 - **Waiting Time (WT) = Start Time – Arrival Time**
 - **Turnaround Time (TAT) = Completion Time – Arrival Time**

FCFS Algorithm

First-Come, First-Served (FCFS) is the simplest CPU scheduling algorithm, where the process that arrives first gets executed first. It follows the FIFO (First In, First Out) principle, like a queue.

- **Waiting Time (WT) = Start Time – Arrival Time**
- **Turnaround Time (TAT) = Completion Time – Arrival Time**
- **TAT = WT + Burst Time**

Working Principle:

1. Processes are added to the ready queue in the order they arrive.
2. CPU picks the first process in the queue and runs it until completion.
3. No preemption – once a process starts, it runs till it finishes.

Priority Based Algorithm

Priority Scheduling is a CPU scheduling algorithm where each process is assigned a priority, and the CPU is allocated to the process with the highest priority.

- Lower numerical value = Higher priority (by default).

- Can be preemptive or non-preemptive

Working Principle:

1. Each process has a priority number.
2. The CPU is assigned to the process with the highest priority.
3. If two processes have the same priority, **FCFS** is used.
4. In **pre-emptive mode**, a process with higher priority can interrupt a running process.

Program Code:

/ Write a program in C to Implement FCFS, SJF and Priority Scheduling Algorithm */*

```
#include <stdio.h>
struct Process {
    int pid;
    int burst;
    int priority;
    int waiting;
    int turnaround;
};

// Function to display results
void display(struct Process p[], int n, const char* algo_name) {
    float total_wait = 0, total_turnaround = 0;
    printf("\n--- %s Scheduling ---\n", algo_name);
    printf("PID\tBurst\tPriority\tWaiting\tTurnaround\n");

    for (int i = 0; i < n; i++) {
        printf("%d\t%d\t%d\t\t%d\t%d\n",
            p[i].pid, p[i].burst, p[i].priority,
            p[i].waiting, p[i].turnaround);

        total_wait += p[i].waiting;
        total_turnaround += p[i].turnaround;
    }

    printf("Average Waiting Time: %.2f\n", total_wait / n);
    printf("Average Turnaround Time: %.2f\n", total_turnaround / n);
}

// FCFS Scheduling
void fcfs(struct Process p[], int n) {
    p[0].waiting = 0;
    for (int i = 1; i < n; i++) {
        p[i].waiting = p[i - 1].waiting + p[i - 1].burst;
    }
    for (int i = 0; i < n; i++) {
```

```
        p[i].turnaround = p[i].waiting + p[i].burst;
    }
}

// SJF Scheduling (Non-preemptive)
void sjf(struct Process p[], int n) {
    struct Process temp;
    // Sort based on burst time
    for (int i = 0; i < n-1; i++)
        for (int j = i+1; j < n; j++)
            if (p[j].burst < p[i].burst) {
                temp = p[i]; p[i] = p[j]; p[j] = temp;
            }

    p[0].waiting = 0;
    for (int i = 1; i < n; i++)
        p[i].waiting = p[i - 1].waiting + p[i - 1].burst;

    for (int i = 0; i < n; i++)
        p[i].turnaround = p[i].waiting + p[i].burst;
}

// Priority Scheduling (Non-preemptive, lower value = higher
priority)
void priority(struct Process p[], int n) {
    struct Process temp;
    // Sort based on priority
    for (int i = 0; i < n-1; i++)
        for (int j = i+1; j < n; j++)
            if (p[j].priority < p[i].priority) {
                temp = p[i]; p[i] = p[j]; p[j] = temp;
            }

    p[0].waiting = 0;
    for (int i = 1; i < n; i++)
        p[i].waiting = p[i - 1].waiting + p[i - 1].burst;

    for (int i = 0; i < n; i++)
        p[i].turnaround = p[i].waiting + p[i].burst;
}

int main() {
    int n;
    struct Process p[10], fcfs_arr[10], sjf_arr[10], prio_arr[10];

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
```

```
p[i].pid = i + 1;
printf("Enter Burst Time and Priority for Process %d: ",
p[i].pid);
scanf("%d%d", &p[i].burst, &p[i].priority);
}

// Copy data to preserve original for each algorithm
for (int i = 0; i < n; i++) {
    fcfs_arr[i] = p[i];
    sjf_arr[i] = p[i];
    prio_arr[i] = p[i];
}

fcfs(fcfs_arr, n);
display(fcfs_arr, n, "FCFS");

sjf(sjf_arr, n);
display(sjf_arr, n, "SJF");

priority(prio_arr, n);
display(prio_arr, n, "Priority");

return 0;
}
```

Output:

```
Enter number of processes: 3
Enter Burst Time and Priority for Process 1: 6 3
Enter Burst Time and Priority for Process 2: 8 1
Enter Burst Time and Priority for Process 3: 7 2
```

--- FCFS Scheduling ---

PID	Burst	Priority	Waiting	Turnaround
1	6	3	0	6
2	8	1	6	14
3	7	2	14	21

Average Waiting Time: 6.67
Average Turnaround Time: 13.67

--- SJF Scheduling ---

PID	Burst	Priority	Waiting	Turnaround
1	6	3	0	6
3	7	2	6	13
2	8	1	13	21

Average Waiting Time: 6.33
Average Turnaround Time: 13.33

--- Priority Scheduling ---

PID	Burst	Priority	Waiting	Turnaround
2	8	1	0	8
3	7	2	8	15
1	6	3	15	21

Average Waiting Time: 7.67

Average Turnaround Time: 14.67

Conclusion:

In this way we implement a program to implement FCFS, SJF and priority based Algorithm